

Number Guessing Game Report

The number guessing game that I have created is dynamic and simple. When you enter into the terminal game, you're introduced to a set of rules describing the game, "1. The Computer Generates a Random Number Between 1-100 (INCLUSIVE) and You Must Guess It In A Certain Number of Attempts. 2. Easy (E) is in 10 Attempts, Medium (M) is in 5 Attempts, Hard (H) is in 3 Attempts. 3. Based On Your Input, The Computer Will Give You Feedback (Too High or Too Low). 4. The Game Will End As Soon As You Get The Correct Number and Will Display The Number of Attempts It Took You." The computer then asks you if you want to play the Number Guessing game or not. You have to either type in Y or N. If you select N, then the game is over. On the other hand, if you type Y, then you are to choose the difficulty you want the game to be. Based on the difficulty you choose, you will be given a set number of attempts: Easy (E) is 10 Attempts; Medium (M) is 5 Attempts; and Hard (H) is 3 Attempts. After choosing you will have to guess the number and based on the number you type in the computer will give you appropriate feedback. For example, if the number is 13 and you type in 7, the computer will state that the number is "Too Low." Vice versa, if you type in 19 then the computer will state that the number is "Too High." If you guess the correct number, then you will be congratulated and will have the number of attempts displayed; however, if you exceed the number of attempts, then you will be given a message of how you have lost and will display what the actual number was.

Now that we understand the overview, it's important to understand the technicality behind the code. I first printed out the introductory messages, and then I used two lists, which are data structures, to store the difficulty levels (e/m/h) and status levels (y/n). I used these lists so I could handle user input. For example, if the user types in "L" in the input box, we check if this string is in the list, if it is not then we prompt the user to put in the appropriate input. After the

user confirms that they want to play, I create a while loop with the condition that status has to equal "y." This is because after each round the user gets prompted if they want to play again, if they enter "y" then the program will start another round. After the while loop we ask the user for the difficulty level and we do the same type of handling as we did before with the appropriate input. After that we call the `number_guessing_game` function which takes the difficulty level as an input. In this function we use the random library and the `randint()` method to generate a number between 1 and 100, inclusive. I use a list to keep track of every number that the user enters, a guess variable with negative infinity, and a dictionary to get how many attempts the user has. The program then prints out how many guesses the user has using that dictionary. Then we have a while loop with a condition of if the attempts exceed the number of attempts allowed and if the guess is the number. This is so that if we have the number then we can exit the loop and conclude that the user won, otherwise the user lost because they exceeded the number of attempts that they have. In the while loop I have a try and except for a `ValueError`. This is because if the user enters a value rather than an int, then the program prompts the user to enter an integer and the round continues to go. In the try statement, the user puts in the guess and it checks if it is too high or too low. After it add the number into the attempts. Then the program shows you how many attempts you have. After you are congratulated or told that you lost. The program asks if you want to play again.

One of the challenges that I faced was being able to handle the user input, and so to fix that I created two lists to check if those values were in that list. If they were not in the list then the user must redo the input. Some improvements could be like adding colors onto the text to make it visibly appealing. I also think that it could be improved by adding end game stats after

all the times that you have played the game. For example, average number of attempts taken to guess the number.